

# **## CODSOFT ARTIFICIAL INTELLIGENCE INTERNSHIP**

## **## Face Detection System**

- **Submitted By:**  
**Neerja Mehar**
- **Internship Domain:**  
**Artificial Intelligence**
- **Organization:**  
**CODSOFT**

## **## Project: Face Detection System**

### **Introduction**

Face Detection is a Computer Vision technique used to identify and locate human faces in digital images.

This project uses OpenCV and Haar Cascade Classifier to detect faces from images and highlight them using bounding

boxes. Face detection is widely used in surveillance system, Attendance system, security applications, and image analysis.

### **Objectives**

- To develop an AI-based Face Detection system.
- To detect human faces from digital images.
- To apply Computer Vision techniques,
- To Understand the work of Haar Cascade System.
- To gain practical experience in Artificial Intelligence and image processing.

### **Features**

- Automatic face detection from images.
- Detects multiple faces in a single image.
- Displays the total number of detected faces.
- Simple and User-friendly implementation.
- Fast and efficient face detection process.

## **System Requirement**

- **Hardware requirement:**

- Processor: Intel i3 or above
- RAM: 4GB or higher
- Storage: 1GB free space

- **Software requirement:**

- Windows 10/11
- Python 3.x
- Visual Studio Code
- OpenCV library

## **Algorithm**

1. Start the program.
2. Load the Haar Cascade face detection model.
3. Read the input image.
4. Convert the image into grayscale format.
5. Detect faces using detectMultiScale().
6. Count the detected faces.
7. Draws rectangles around detected faces.
8. Display the output image.
9. Show the total number of faces detected.
10. End the program

## Source Code

```
import cv2

# Load Haar cascade
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades +
    "haarcascade_frontalFace_default.xml"
)

# Load Image
image = cv2.imread("person.jpg")

# Covert Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect Faces
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=10,
    minSize=(100,100)
)

print("Faces Detected:" , len(faces))
print(faces)

# Draw rectangles around faces
for (x, y, w, h) in faces:
```

```
cv2.rectangle(image, (x, y), (x+w, y+h), (255, 0, 0),  
2)
```

```
# Show output  
cv2.imshow("Face Detection", image)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

### **Sample Output**

```
Faces Detected: 4  
[[ 624 208 592 592]  
 [1996 611 498 498]  
 [1704 868 430 430]  
 [1130 846 549 549]]
```

### **Learning Outcomes**

- Learned the fundamentals of face detection.
- Gained practical experience with OpenCv.
- Understood the working of Haar Cascade Classifier.
- Improved knowledge of Computer Vision concepts.
- Enhanced Python Programming and debugging skills.

### **Future Enhancement**

- Real-time face detection using webcam.

- Face recognition with identity matching.
- Attendance management system integration.
- Emotion detection from facial expression.
- Deep Learning-based face detection for improved accuracy.

## **Conclusion**

The face detection system is successfully detects human from images using Opencv and Haar Cascade Classifier. The project demonstrates the practical implementation of Computer Vision techniques for identifying faces in digital images. It provides a strong foundation for advanced application such as face recognitions, surveillance system, and smart attendance solution.